

Go Game Project

Hoàng Nguyên Anh, Nguyễn Phan Minh Nhật

Last edited day: November 19, 2025

Overview

The project aims to develop a digital version of the board game Go (Weiqi/Baduk) using C++. The application focuses on a smooth, minimalist 2-player experience, allowing two users to play against each other on the same device. The project emphasizes strict adherence to the rules of Go, including stone placement, capturing mechanics, and liberte management, while providing a lightweight and responsive user interface.

Contents

1	Requirements	2
1.1	Game Modes	2
1.2	User Interface (UI)	2
1.3	Board Representation	3
1.4	Game Logic	3
1.5	Game State Management	3
1.6	Technology Stack	4
2	Design Document	5
2.1	Overview	5
2.2	Processes	5
2.2.1	Mouse Event Processing	5
2.2.2	Move Validation Flow	5
2.3	User Interface Design	5
2.3.1	Theme	5
3	Implementation Details	7
3.1	File Organization	7
3.2	Core Algorithms	7
3.2.1	Flood Fill for Liberties	7
4	Testing	8
4.1	Methodology	8
4.2	Known Issues	8
5	Future Improvements	9

Chapter 1

Requirements

1.1 Game Modes

- **2-Player Mode (PvP)**
 - Two players compete against each other on the same device.
 - The game manages turns automatically, alternating between Player 1 (Black stones) and Player 2 (White stones).
 - visual indicators show whose turn it is currently.

1.2 User Interface (UI)

- **The Board:** A graphical representation of a 19×19 Go board (grid intersections).
- **Start Menu:**
 - **New Game:** Starts a fresh 2-player match.
 - **Load Game:** Opens a menu to select previously saved game states.
 - **Settings:** Opens audio and visual customization options.
 - **Quit:** Exits the application.
- **In-Game UI:**
 - **Turn Indicator:** Displays "Black's Turn" or "White's Turn".
 - **Capture Counter:** Displays the number of prisoners (stones captured) by each player.
 - **Control Buttons:**
 - * **Undo:** Reverts the last move.
 - * **Redo:** Re-applies the last undone move.
 - * **Pass:** Allows a player to pass their turn (crucial for game ending in Go).
 - * **Save:** Saves the current board state to a file.
 - * **Home:** Returns to the main menu.

1.3 Board Representation

- The board is rendered using the SFML graphical library.
- Stones are circular bitmaps or vector graphics placed on the intersections of the grid lines, not inside the squares.
- **Visual Feedback:**
 - A "ghost stone" follows the mouse cursor to show where a stone will be placed if clicked.
 - Invalid moves (such as placing a stone on top of another) show a visual error indicator (e.g., red tint).
 - The most recently placed stone is marked with a small dot or marker to help players track the game flow.

1.4 Game Logic

The core engine implements the standard rules of Go:

- **Placement:** Players take turns placing a stone on an empty intersection.
- **Liberties and Capturing:**
 - The system calculates the "liberties" (adjacent empty points) of every group of stones after every move.
 - If a move reduces an opponent group's liberties to zero, that group is removed from the board and added to the capturing player's prisoner count.
- **Suicide Rule:** A player cannot place a stone such that their own group would immediately have zero liberties, unless that move captures opponent stones.
- **Ko Rule:** The system prevents a move that recreates the exact board state from the previous turn (preventing infinite loops).
- **Game End:** The game ends when both players pass consecutively.

1.5 Game State Management

- The game state is stored using a 2D array representing the grid intersections (0 = Empty, 1 = Black, 2 = White).
- **History Stack:** Two stacks are maintained to handle Undo/Redo functionality. Each stack node contains the full board state and the current capture counts.
- **Save/Load:** The game serializes the board array, current turn, and capture counts into a text/binary file for persistent storage.

1.6 Technology Stack

- **Language:** C++ for core logic and state management.
- **Graphics:** SFML (Simple and Fast Multimedia Library) for rendering the board, stones, and handling mouse input events.
- **Architecture:** Object-Oriented Design separating 'GameEngine', 'RenderingSystem', and 'InputHandler'.

Chapter 2

Design Document

2.1 Overview

The architecture follows a loop-based game cycle: Input Processing → Logic Update → Rendering.

2.2 Processes

2.2.1 Mouse Event Processing

When the user clicks the mouse:

1. The pixel coordinates of the mouse are converted to board grid coordinates (x, y) .
2. The system checks if the click is within the board boundaries.
3. If valid, the coordinate is sent to the Game Logic Validator.

2.2.2 Move Validation Flow

1. **Occupancy Check:** Is the intersection (x, y) empty?
2. **Liberty Check:** Does placing the stone here result in 0 liberties (Suicide)?
3. **Capture Check:** Does placing the stone capture adjacent opponent stones? (If yes, Suicide rule is ignored).
4. **Ko Check:** Does the resulting board match the history immediately prior?
5. If all checks pass, the move is committed, the board is updated, and the turn switches.

2.3 User Interface Design

2.3.1 Theme

The game utilizes a classic aesthetic:

- **Board:** A warm wooden texture resembling a traditional Kaya board.
- **Stones:** Slate and Shell textures for Black and White stones respectively.

- **Modern Overlay:** Menus use a semi-transparent dark overlay with flat, minimalist buttons to ensure text readability.

Figure 2.1: Mockup of the Go Board Interface (Place image here)

Chapter 3

Implementation Details

3.1 File Organization

The project is structured into header (.h) and source (.cpp) files:

- `main.cpp`: Entry point, initializes the window and game loop.
- `Game.h/cpp`: Manages the main loop and switches between Menu and Gameplay states.
- `Board.h/cpp`: Stores the 19x19 grid data and handles rendering of the grid.
- `Rules.h/cpp`: Contains the algorithms for counting liberties and detecting captures (Flood Fill algorithm).
- `UI.h/cpp`: Handles button clicks and menu rendering.

3.2 Core Algorithms

3.2.1 Flood Fill for Liberties

To determine captures, a recursive flood-fill algorithm is used. When a stone is placed, the algorithm checks all 4 orthogonal neighbors. If a neighbor belongs to the opponent, the algorithm traverses that group to count its total liberties. If the count is 0, the group is removed.

Chapter 4

Testing

4.1 Methodology

Testing was conducted primarily through manual playtesting in 2-player mode.

- **Legal Moves:** Verified that stones snap to intersections correctly.
- **Capture Logic:** Tested complex shapes (e.g., "Ladder", "Snapback") to ensure stones are removed only when completely surrounded.
- **Suicide Rule:** Attempted to place stones in spots with no liberties to ensure the game prevents the move.
- **Ko Rule:** Created repetitive capture scenarios to verify the game prevents infinite loops.

4.2 Known Issues

- In extremely large group captures, there is a slight frame drop due to the recursive checking of the entire board.
- Board scaling issues on non-standard aspect ratios (currently fixed to 1:1 aspect for the board area).

Chapter 5

Future Improvements

1. **AI Opponent:** Implement a Monte Carlo Tree Search (MCTS) or Minimax AI to allow single-player practice.
2. **Scoring System:** Implement an automatic territory counting algorithm (e.g., Chinese or Japanese scoring rules) to calculate the winner automatically at the end of the game.
3. **Network Play:** Add socket support to play against an opponent over LAN or Internet.
4. **Timer:** Add a "Byoyomi" timer system for competitive play.